



Physics-based Reward Design for Safe Reinforcement Learning Control

M.Deng^a, H.Lin^a K.Dasgupta^a

^aDepartment of Physics, McGill University, Montreal, Québec, Canada

Abstract

Reinforcement learning (RL) has recently been applied to the control of complex dynamical systems. However, standard RL formulations do not explicitly account for underlying physical laws, which can lead to oscillatory or non-smooth trajectories. Moreover, due to the exploratory nature of RL, a robot may easily visit unsafe states during training. In this project, we propose a physics-based reward shaping method that incorporates system energy directly into the learning process, resulting in faster convergence compared to the baseline reward. In addition, we employ a Control Barrier Function (CBF) formulation as a safety filter that modifies unsafe RL actions into safe ones while keeping them close to the original proposal. This ensures safety during training without overly restricting the policy. We compare our CBF-based approach with other safe RL methods and show that the CBF filter achieves faster convergence and zero safety violations when activated.

1 Introduction

Controlling dynamical systems has wide applications, from controlling thermal energy storage [1] to control of complex quantum system [2]. However, controlling complex dynamic system efficiently and reliably remains a central challenge in physics and engineering [3]. One recent solution is to use reinforcement learning. It is a method that teaches machines to achieve specific objectives by providing rewards that indicate performance quality at each time step [4]. Conventional reinforcement learning approaches typically rely on heuristic[5] or sparse reward signals[6]. This can lead to trajectories that are energetically inefficient, oscillatory, or otherwise non-physical[7]. For systems where energy expenditure, smooth motion, or adherence to physical constraints is critical, purely data-driven approaches[8] may produce solutions that are difficult to interpret or implement in real-world settings. There is thus a need for control methods that combine the flexibility of learning-based approaches with the rigor of physical principles.

Recent work in reinforcement learning has explored shaping rewards or incorporating domain knowledge to guide agents toward more efficient or safe behaviours. Techniques include penalizing control effort[9], using dense feedback based on configuration or velocity, and occasionally incorporating simplified physical models. While these approaches improve learning performance in some scenarios, they often treat physical laws indirectly and do not fully exploit energy conservation, potential energy landscapes, or other fundamental properties of the system. As a result, trajectories may still be suboptimal in terms of energy usage, smoothness, or interpretability. A systematic, physics-informed design of the learning objective is still lacking in current methods.

Safety during exploration is another major challenge in RL [10]. Since agents initially lack knowledge of the environment, they must try different actions to learn how the system responds. This exploration process can cause violations of safety constraints, such as exceeding allowable velocities or entering hazardous regions of the state space. Ensuring safety while maintaining learning efficiency is therefore essential for both simulated and real-world control tasks.

We propose to develop a reinforcement learning framework that explicitly integrates physical principles into the reward design to produce efficient and interpretable trajectories. Rewards will be constructed from fundamental quantities such as potential energy, kinetic energy, and mechanical work, allowing the agent to learn behaviours that align with the underlying physics. By leveraging energy conservation and smoothness constraints, the framework will encourage minimal energy expenditure, reduce oscillatory motion, and improve learning efficiency.

Furthermore, to ensure safety during training, we incorporate Control Barrier Functions (CBFs) as a real-time safety filter. The CBF modifies unsafe actions into safe ones while keeping them as close as possible to the original RL proposals. This prevents the agent from violating safety constraints during exploration without overly restricting learning dynamics. These two approaches is general and can be applied to a wide range of dynamical systems. Offering a principled way to combine classical physics with machine learning for the design of energy-efficient, safe, and physically interpretable control strategies.

The remainder of the paper is organized as follows. Section 2.1 introduces reinforcement learning terminology and problem formulation. Section 2.3 presents the theory of Control Barrier Functions. Section 2.2 introduces the RL policies we used in this project. Section 2 describes the inverted-pendulum environment used for simulation. Section 3.2 details the physics-based reward design and evaluates the resulting performance. Section 3.3 compares different RL policies under safety constraints.

2 Preliminaries

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a framework for teaching an agent (ex, a robot or a software program) to make decisions by interacting with an environment [4]. Figure 1 illustrates how the agent and environment interact with each other. At every step of interaction, the agent observes the current state of the environment, chooses an action, and then receives a reward indicating how good or bad that action was. Over time, the agent learns to select actions that lead to higher rewards.

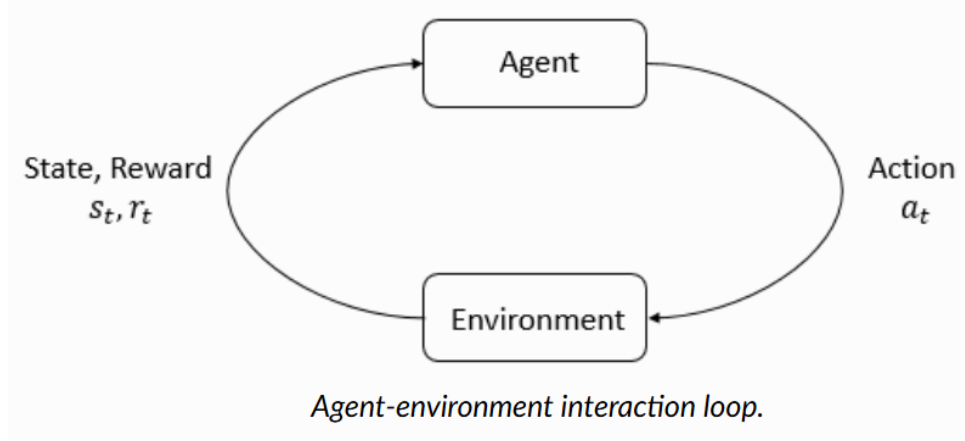


Figure 1: Agent-environment interaction loop: Firstly, the agent observes the current state of the environment. Secondly, it chooses an action based on that observation. Thirdly, the environment transitions to a new state based on the action. Lastly, the agent receives a scalar reward.

A policy π describes the agent’s behavior. $\pi(a \mid s)$ is the probability that the agent chooses action a when it is in state s .

The value function $V^\pi(s)$ for a policy π measures how much reward the agent can expect to accumulate in the future when starting from state s and following π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right], \quad (1)$$

where s_0 is the initial state, $\gamma \in [0, 1]$ is the discount factor, which balances the importance of immediate rewards versus future rewards. $r(s_t, a_t)$ is the reward function, which outputs a scalar value indicating the immediate quality of taking action a in state s .

The objective in RL is to find an optimal policy π^* that maximizes $V^\pi(s)$ for all states.

2.2 Reinforcement Learning Policy

In this paper, we use Proximal Policy Optimization (PPO) as our baseline reinforcement learning (RL) policy without any safety-aware constraints. We also consider two safety-aware RL algorithms: Proximal Policy Optimization–Lagrangian (PPO-Lagrangian) and Constrained Policy Optimization (CPO). Below, we briefly describe how each policy works.

Proximal Policy Optimization (PPO)

PPO [11] is a widely used RL algorithm designed to train a policy while maintaining stable learning. A simplified PPO training loop is shown below:

1. Run the current policy in the environment and collect experience in the form of state-action-reward tuples (s, a, r) .
2. Estimate the value function $V^\pi(s)$ that tells how good each state is and how the policy can improve.
3. Update the policy by clipping the magnitude of update steps. Prevent the new policy from changing too drastically, and keep learning stable.
4. Update the value function according to the observed return.
5. Repeat the loop until the policy converges.

PPO repeatedly improves the policy while making sure each update stays close to the previous one, avoiding large, unstable jumps.

Proximal Policy Optimization–Lagrangian (PPO-Lagrangian)

PPO-Lagrangian [12] builds on PPO by adding safety constraints in the form of a cost function. For example, if we want to limit the agent’s velocity, we can define a cost such as $c = \max(0, v - v_{\max})$, which penalizes the agent whenever it exceeds the maximum allowed velocity.

The reward is modified as

$$r = r_{\text{old}} - \lambda c, \quad (2)$$

where λ is a Lagrange multiplier that determines how strongly violations of the safety constraint are penalized.

The goal of PPO-Lagrangian is to maximize task performance, and automatically adjusts λ so that safety constraints are satisfied during training.

Constrained Policy Optimization (CPO)

Constrained Policy Optimization (CPO) [13] is another safety-aware RL method. Like PPO, it performs policy updates that remain close to the current policy to ensure stable learning. However, CPO explicitly enforces that the total estimated cost of the updated policy must remain below a specified maximum allowable cost. The algorithm rejects any policy update that would cause the expected cost to exceed this safety threshold. If a proposed update violates the constraint, CPO performs a recovery step to project the policy back into a safe region.

Policy Evaluation

During training, the agent occasionally takes random actions to explore unknown states. The exploration process helps the agent discover new strategies and prevents it from getting stuck in poor behaviours.

During policy evaluation, no random actions are taken. The agent follows the learned policy, allowing us to measure the true performance and safety awareness of the trained policy.

2.3 Control Barrier Function

In the Control Barrier Function (CBF) framework [14], let $\mathbf{q}$ denote the current state of the system, and let $h(\mathbf{q})$ be a CBF. The state $\mathbf{q}$ is considered safe when $h(\mathbf{q}) \geq 0$. Safety is maintained if

$$\dot{h}(\mathbf{q}, \mathbf{u}) \geq -\alpha h(\mathbf{q}), \quad (3)$$

where $\mathbf{u}$ is the control input and $\alpha > 0$ is a constant. Enforcing the stricter condition $\dot{h}(\mathbf{q}, \mathbf{u}) \geq 0$ would require the system to always move deeper into the safe region, which is often overly conservative. The relaxed constraint in Eq. 3 allows the state to approach the boundary of the safe set, as long as it does not leave it.

We assume that the system dynamics are given by

$$\dot{\mathbf{q}} = f(\mathbf{q}) + g(\mathbf{q}) \mathbf{u}, \quad (4)$$

where $f(\mathbf{q})$ describes the natural (uncontrolled) evolution of the system, and $g(\mathbf{q})$ characterizes how the control input affects the state. Using the chain rule, the time derivative of $h(\mathbf{q})$ becomes

$$\dot{h}(\mathbf{q}, \mathbf{u}) = \nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \quad (5)$$

$$= \nabla h(\mathbf{q})^\top f(\mathbf{q}) + \nabla h(\mathbf{q})^\top g(\mathbf{q}) \mathbf{u}. \quad (6)$$

Substituting this expression into the safety condition (3) yields a linear constraint on the control input:

$$\nabla h(\mathbf{q})^\top g(\mathbf{q}) \mathbf{u} \geq -\nabla h(\mathbf{q})^\top f(\mathbf{q}) - \alpha h(\mathbf{q}). \quad (7)$$

We can write this compactly as

$$A(\mathbf{q}) \mathbf{u} \geq b(\mathbf{q}), \quad (8)$$

where

$$A(\mathbf{q}) = \nabla h(\mathbf{q})^\top g(\mathbf{q}), \quad b(\mathbf{q}) = -\nabla h(\mathbf{q})^\top f(\mathbf{q}) - \alpha h(\mathbf{q}).$$

To obtain a safe control action $\mathbf{u}_{\text{safe}}$ that satisfies the CBF constraint while remaining as close as possible to the RL-proposed action $\mathbf{u}_{\text{RL}}$, we solve the following quadratic program (QP):

$$\begin{aligned} \mathbf{u}_{\text{safe}}^* = \arg \min_{\mathbf{u}} \quad & \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{RL}}\|_2^2 \\ \text{subject to} \quad & A(\mathbf{q}) \mathbf{u} \geq b(\mathbf{q}), \\ & \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}. \end{aligned} \quad (9)$$

Since the objective function is quadratic and the constraints are linear, the resulting optimization problem is a CBF-QP (Control Barrier Function Quadratic Program). It is convex and can be solved efficiently in real time, making it suitable for safety-critical control applications.

3 Methodology & Results

3.1 Testing Environment

We test both physics-based reward shaping methods and safe RL policies in the inverted pendulum environment [15]. This is a 1D environment in which a cart moves along a horizontal

line while a pole with uniform mass is hinged to the top of the cart. A snapshot of the system is shown in Figure 2. The agent’s action is $a \in [-3, 3]$, representing the horizontal force applied to the cart. A positive action pushes the cart to the right, and a negative action pushes it to the left.

The state of the system is

$$\mathbf{q} = [x, \dot{x}, \theta, \dot{\theta}]^T,$$

where x is the position of the cart, θ is the angle of the pole measured from the vertical, $\dot{x}$ is the cart’s velocity, and $\dot{\theta}$ is the angular velocity of the pole.

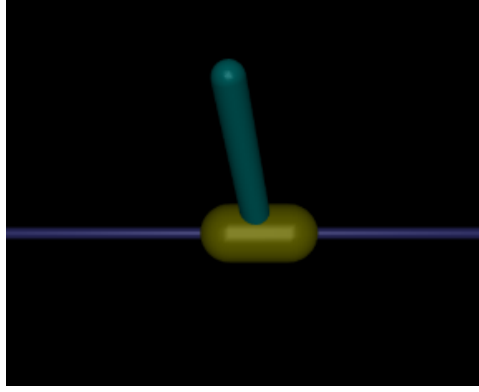


Figure 2: A snapshot of the inverted pendulum environment. The cart is moving along a horizontal line, and a pole with uniform mass is hinged to the top of the cart

The objective of the agent is to keep the pole upright (i.e., within a small angular range) for as long as possible. The environment provides a reward of +1 at each timestep if $|\theta| < 0.2$. The episode terminates when the pole falls too far, when $|\theta| \geq 0.2$. The environment also truncates once it reaches the maximum duration of 1000 timesteps.

During training, one full interaction sequence between the agent and the environment is called an episode. An episode ends either when the environment terminates due to failure or when it truncates after reaching the timestep limit.

3.2 Physics-Based Reward Shaping

The total energy of the inverted pendulum system is

$$E = \frac{1}{2} \left[(M + m)\dot{x}^2 + m\dot{x}l\dot{\theta} + \frac{1}{3}ml^2\dot{\theta}^2 + mlg(1 - \cos(\theta)) \right],$$

where M is the mass of the cart, m is the mass of the pole, and l is the pole length. The potential energy is chosen to be zero when the pole is upright. To keep the pole stable and near the upright position, we want the system’s total energy to remain small.

To encourage this, we introduce a physics-based reward-shaping term. The shaped reward is defined as

$$r(\mathbf{q}) = 1 - \gamma E, \quad (10)$$

where γ is a positive scalar chosen so that the reward remains non-negative. Intuitively, this reward gives the agent a higher score when the system energy is low, pushing it toward states that correspond to better balance.

Experiment Result

We compare two reward designs in the inverted pendulum environment:

- the standard reward (i.e., +1 per timestep when the pole stays within the allowed angle)
- the shaped reward defined in Eq. 10.

Using each reward design, we train a set of 10 PPO policies with different random seeds. The random seed affects factors such as the initial state distribution and the randomness during exploration. Figure 4 shows the average episode length over training timesteps for each set of policies. We use episode length instead of reward return to allow a fair comparison between reward functions.

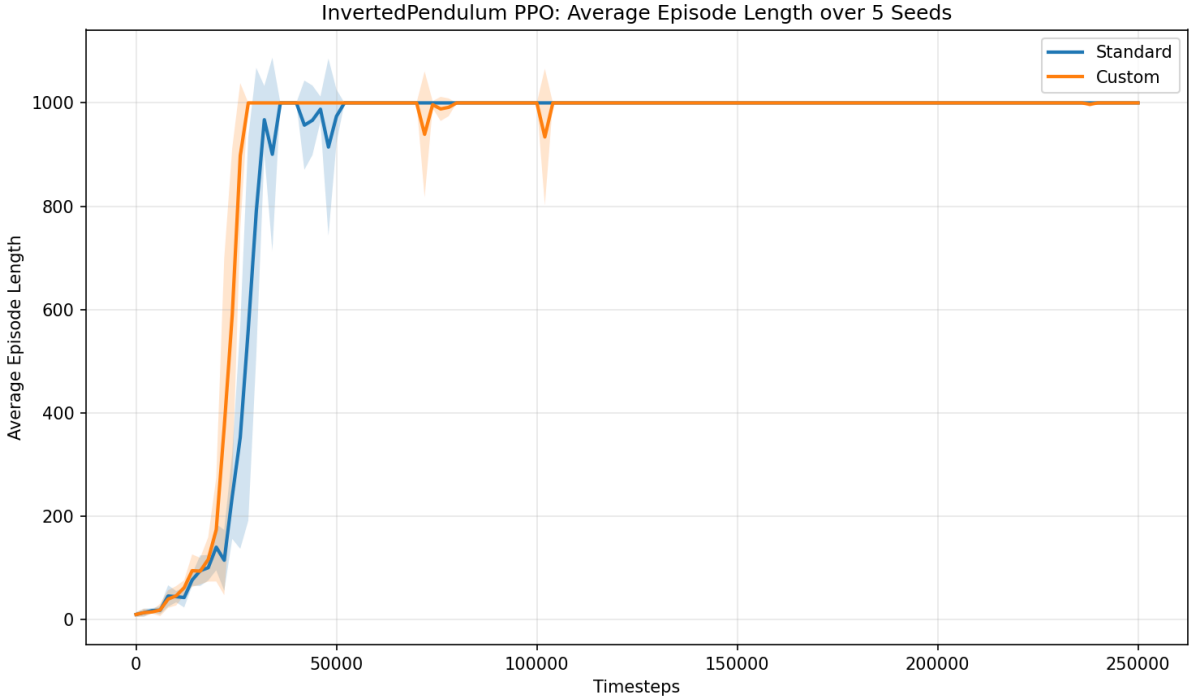


Figure 3: Average episode length during training for PPO with standard reward and with energy-based reward shaping. The shadowed area is the standard deviation of the average episode length at each timestep.

From the plot, policies trained with the shaped reward reach the maximum episode length of 1000 timesteps slightly faster than those trained with the standard reward. With custom reward, the policy achieves maximum return with 25000 ± 2000 timesteps. While for standard reward, it takes 29000 ± 3000 timesteps. We evaluate all trained policies over 60 evaluation episodes. Both sets achieve an average episode length of 1000.0 ± 0.0 , demonstrating that reward shaping accelerates learning without changing the final performance.

3.3 Safe Reinforcement Learning

Control Barrier Function Formulation

We set the safety constraint on the cart’s position to be $x \in [-x_{\max}, x_{\max}]$. The system is considered unsafe once the cart moves outside this region, and the episode terminates immediately.

To construct the CBF for the inverted pendulum system, we first derive the system dynamics (4) using the Euler–Lagrange equation with the Lagrangian

$$L = \frac{1}{2} \left[(M + m)\dot{x}^2 + m\dot{x}l\dot{\theta}\cos\theta + \frac{1}{3}ml^2\dot{\theta}^2 - mlg(1 - \cos\theta) \right]. \quad (11)$$

The Euler–Lagrange equations with respect to x and θ are:

$$(M + m)\ddot{x} + \frac{1}{2}ml\ddot{\theta}\cos\theta - \frac{1}{2}ml\sin\theta\dot{\theta}^2 = u, \quad (12)$$

$$\frac{1}{2}ml\ddot{x}\cos\theta + \frac{1}{3}ml^2\ddot{\theta} + \frac{1}{2}mgl\sin\theta = 0. \quad (13)$$

Solving for $\ddot{x}$ and $\ddot{\theta}$ yields the state-space dynamics

$$\dot{\mathbf{q}} = \underbrace{\begin{bmatrix} \dot{x} \\ \frac{\frac{1}{2}ml\dot{\theta}^2\sin\theta + \frac{3}{4}mg\sin\theta\cos\theta}{M + m - \frac{3}{4}m\cos^2\theta} \\ \dot{\theta} \\ -\frac{\frac{3}{2}g\sin\theta - \frac{3}{2l}\cos\theta\frac{\frac{1}{2}ml\dot{\theta}^2\sin\theta + \frac{3}{4}mg\sin\theta\cos\theta}{M + m - \frac{3}{4}m\cos^2\theta}}{2l(M + m - \frac{3}{4}m\cos^2\theta)} \end{bmatrix}}_{f(\mathbf{q})} + \underbrace{\begin{bmatrix} 0 \\ 1 \\ \frac{1}{M + m - \frac{3}{4}m\cos^2\theta} \\ 0 \\ \frac{3\cos\theta}{2l(M + m - \frac{3}{4}m\cos^2\theta)} \end{bmatrix}}_{g(\mathbf{q})} u. \quad (14)$$

The CBF associated with the constraint $|x| \leq x_{\max}$ is defined as

$$h(x) = \begin{cases} x_{\max} - x, & x \geq 0, \\ x_{\max} + x, & x < 0. \end{cases} \quad (15)$$

The first derivative $\dot{h} = \pm\dot{x}$ does not depend on the control input u , so we must use a higher-order CBF (HOCBF). The HOCBF sequence is

$$\Psi_0 = h(\mathbf{q}), \quad (16)$$

$$\Psi_r = \dot{\Psi}_{r-1} + \alpha_r \Psi_{r-1} \geq 0. \quad (17)$$

For this system, the control input appears at the second derivative of $h(x)$, which depends on $\ddot{x}$. Thus the second-order HOCBF is

$$\Psi_2 = \ddot{h} + (\alpha_2 + \alpha_1)\dot{h} + \alpha_1\alpha_2h \geq 0. \quad (18)$$

Substituting $\dot{h} = \pm\dot{x}$ and $\ddot{h} = \pm\ddot{x}$, we obtain:

$$\pm\ddot{x} + (\alpha_2 + \alpha_1)(\pm\dot{x}) + \alpha_1\alpha_2(x_{\max} \pm x) \geq 0. \quad (19)$$

Using the dynamics, $\ddot{x} = f_2 + g_2u$, we obtain the linear constraint on u :

$$\pm(f_2 + g_2u) + (\alpha_2 + \alpha_1)(\pm f_1) + \alpha_1\alpha_2(x_{\max} \pm x) \geq 0. \quad (20)$$

Define

$$A = \pm g_2, \quad b = \mp f_2 - (\alpha_2 + \alpha_1)(\pm f_1) - \alpha_1\alpha_2(x_{\max} \pm x),$$

so the CBF constraint becomes

$$Au \geq b,$$

where f_n and g_n denote the n -th components of $f(\mathbf{q})$ and $g(\mathbf{q})$.

Cost Function Formulation

For both PPO-Lagrangian and CPO, we must define a cost function that penalizes approaching or violating the safety boundary. We use a logarithmic barrier function:

$$c = -\mu \ln\left(1 - \left(\frac{x}{x_{\max}}\right)^2\right), \quad (21)$$

where μ controls the penalty magnitude. This cost function is smooth, has well-behaved gradients and Hessians (the second derivative of the function), and approaches infinity as $|x| \rightarrow x_{\max}$, strongly discouraging unsafe states.

Experiment Result

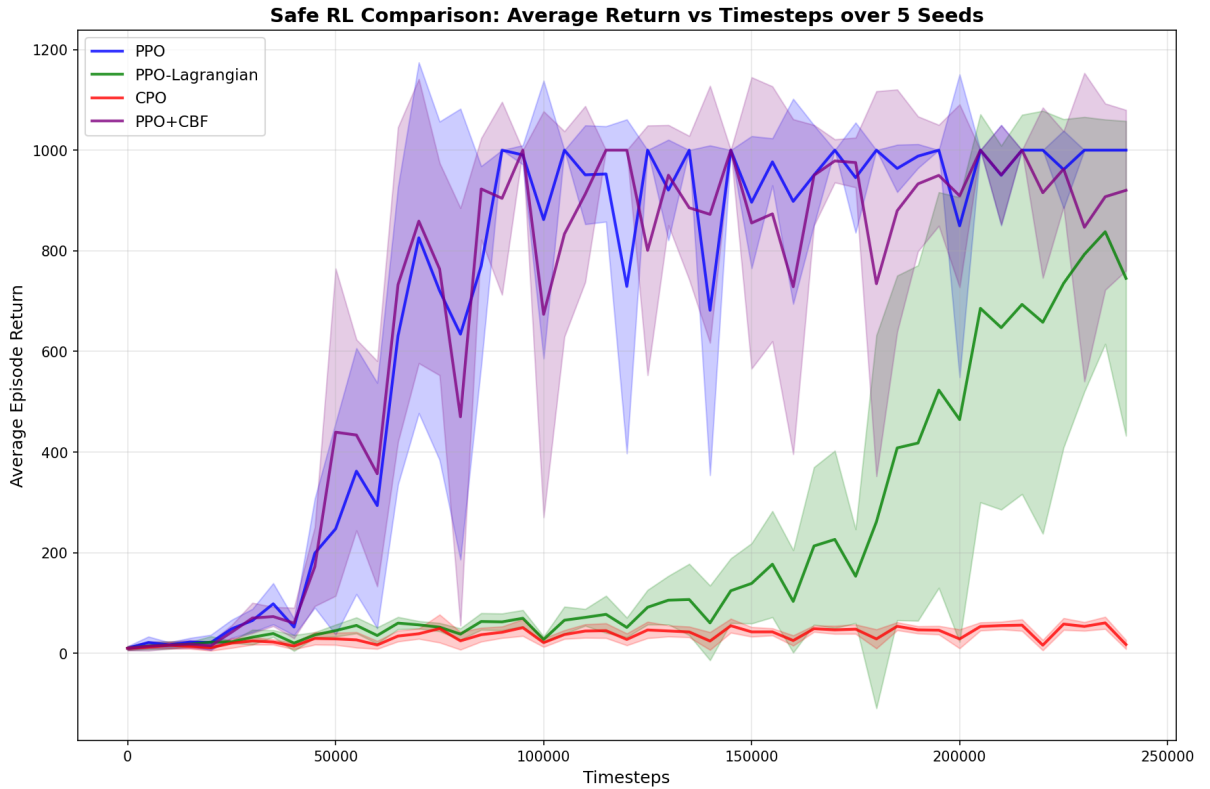


Figure 4: Training performance of different safe RL methods. The shadowed area is the standard deviation of the average episode return. PPO+CBF and vanilla have similar convergence speed, while PPO-Lagrangian and CPO converge much slower.

We train four sets of policies in the inverted pendulum environment: PPO (baseline), PPO-Lagrangian, CPO, and PPO+CBF (PPO filtered through a CBF-QP).

All policies use the standard reward of +1 per timestep while the pole remains upright. Each set contains five policies trained with different random seeds. The training curves are shown in Figure 4.

PPO+CBF converges at roughly the same speed as vanilla PPO, while PPO-Lagrangian and CPO converge significantly more slowly. PPO-Lagrangian adapts the Lagrange multiplier during training, which continuously shifts the optimization target and slows convergence. CPO enforces strict safety during every update, resulting in very small update steps and therefore much slower learning.

Policy	Timesteps to achieve max return	Violation rate (training)	Violation rate (evaluation)	Average return (evaluation)
PPO	84000 ± 5000	$16 \pm 5\%$	$16 \pm 12\%$	930 ± 140
PPO-Lagrangian	-	$0.3 \pm 0.2\%$	$7 \pm 3\%$	890 ± 150
CPO	-	$0.03 \pm 0.03\%$	$0.0 \pm 0.0\%$	57 ± 5
PPO+CBF	86000 ± 3000	$0.0 \pm 0.0\%$	$10 \pm 6\%$	930 ± 130

Table 1: Safety violation rates and evaluation returns for different safe RL methods. PPO and PPO+CBF take roughly the same timesteps to achieve the maximum return. While PPO-Lagrangian and CPO never achieve the maximum return during the entire training process.

Table 1 summarizes the violation rate (the rate that cart exceeded $x_{\max}$) and the average return during evaluation.

During training, PPO+CBF incurs zero violations, strictly satisfying the safety constraint. PPO-Lagrangian and CPO also achieve much lower violation rates than vanilla PPO, though some violations still occur.

During evaluation, the CBF filter is not active. PPO and PPO+CBF achieve similar returns, and PPO+CBF exhibits a slightly lower violation rate. PPO-Lagrangian and CPO achieve lower returns but are significantly safer. Their lower returns arise because neither method fully converged during training.

4 Conclusion

In this project, we explore both physics energy-based reward shaping and Control Barrier Functions (CBF) used as a safety filter. Our experiments in the inverted pendulum environment show that energy-based reward shaping has slightly faster convergence compared to the standard reward function. Moreover, PPO combined with a CBF filter (PPO+CBF) achieves a zero violation rate during training, demonstrating strict enforcement of the safety constraint. However, during evaluation, when the CBF filter is not applied, the trained policies still have safety violations.

In the inverted pendulum system, where the objective is to maintain the pole in an upright and stable configuration, it is natural to incorporate the system’s physical energy into the RL reward. Minimal total energy corresponds to a state close to equilibrium, and large deviations in kinetic or potential energy indicate instability. By embedding this physical principle directly into the reward, the agent learns how its actions influence the system’s energy landscape and can use this information to converge toward an optimal control policy more efficiently than with heuristic rewards alone.

By deriving the system dynamics analytically and applying CBF to encode position constraints, we use the underlying equations of motion to enforce safety. The CBF operates as a real-time safety filter, modifying unsafe RL actions into safe ones while minimally altering the agent’s intended behavior. Because the filter is grounded in the true system dynamics, it provides a robust and physically interpretable method for ensuring that the controller respects safety limits.

Future work includes extending to more complex environments and incorporating CBF-based reward shaping. The goal is to guide the learning process so that the resulting policies satisfy safety constraints, even in the absence of an explicit CBF filter at deployment time.

References

- [1] Donghun Kim et al. *Site demonstration and performance evaluation of MPC for a large chiller plant with TES for renewable energy integration and grid decarbonization*. In: *Applied Energy* 321 (2022), p. 119343. DOI: 10.1016/j.apenergy.2022.119343.
- [2] Zheng An et al. *Quantum optimal control of multilevel dissipative quantum systems with reinforcement learning*. In: *Physical Review A* 103.1 (2021), p. 012404. DOI: 10.1103/PhysRevA.103.012404.
- [3] Xin Zhang et al. *Foresight and relaxation enable efficient control of nonlinear complex systems*. In: *Physical Review Research* 5.3 (2023), p. 033138. DOI: 10.1103/PhysRevResearch.5.033138.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [5] Zhijie Zhang et al. *Deep reinforcement learning for path planning of autonomous mobile robots in complicated environments*. In: *Complex & Intelligent Systems* 11.6 (2025), p. 277. DOI: 10.1007/s40747-025-01906-9. URL: <https://doi.org/10.1007/s40747-025-01906-9>.
- [6] Sreehari Rammohan et al. *Value-Based Reinforcement Learning for Continuous Control Robotic Manipulation in Multi-Task Sparse Reward Settings*. In: *arXiv preprint arXiv:2107.13356* (2021). URL: <https://arxiv.org/abs/2107.13356>.
- [7] Siddharth Mysore et al. “Regularizing Action Policies for Smooth Control with Reinforcement Learning”. In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*. 2021, pp. 1616–1623. DOI: 10.1109/ICRA48506.2021.9561356. URL: https://cs-people.bu.edu/rmancuso/files/papers/ICRA21_1616_FI.pdf.
- [8] Alejandro Escontrela et al. “Adversarial Motion Priors Make Good Substitutes for Complex Reward Functions”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. arXiv preprint arXiv:2203.15103. 2022. URL: https://xbpeng.github.io/projects/AMP_Locomotion/AMP_Locomotion_2022.pdf?utm_.
- [9] Xibao Xu, Yushen Chen, and Chengchao Bai. *Deep Reinforcement Learning-Based Accurate Control of Planetary Soft Landing*. In: *Sensors* 21.23 (2021), p. 8161. DOI: 10.3390/s21238161. URL: <https://doi.org/10.3390/s21238161>.
- [10] Ankita Kushwaha et al. *A Survey of Safe Reinforcement Learning and Constrained MDPs: A Technical Survey on Single-Agent and Multi-Agent Safety*. In: *arXiv preprint arXiv:2505.17342* (2025). URL: <https://arxiv.org/abs/2505.17342>.
- [11] John Schulman et al. *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347. 2017. URL: <https://arxiv.org/abs/1707.06347>.
- [12] Alex Ray, Joshua Achiam, and Dario Amodei. *Benchmarking Safe Exploration in Deep Reinforcement Learning*. Tech. rep. arXiv preprint arXiv:1901.08610. OpenAI, 2019. URL: <https://cdn.openai.com/safexp-short.pdf>.
- [13] Joshua Achiam et al. “Constrained Policy Optimization”. In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. Sydney, Australia: PMLR, 2017, pp. 22–31. URL: <https://proceedings.mlr.press/v70/achiam17a.html>.
- [14] Aaron D. Ames, Jessy W. Grizzle, and Paulo Tabuada. “Control Barrier Function Based Quadratic Programs for Safety Critical Systems”. In: *2014 American Control Conference (ACC)*. 2014, pp. 2928–2935. DOI: 10.1109/ACC.2014.6859152.
- [15] Farama Foundation. *Inverted Pendulum (MuJoCo) Environment*. https://gymnasium.farama.org/environments/mujoco/inverted_pendulum/. 2023.